

Serverless Function Consolidation Analysis

Serverless Function Consolidation Analysis (Read-only)

1. Executive Summary
2. Current Serverless-Function Count
3. Current API and Serverless Directory Tree
4. Complete Serverless-Function Inventory
5. Frontend-to-API Dependency Map
6. Duplicate and Overlapping Function Analysis
7. Functions That Can Be Merged
8. Functions That Should Share Logic But Remain Separate
9. Functions That Must Not Be Merged
10. Separate Route Merging From Code Deduplication
11. Recommended Final API Structure
12. Vercel Serverless Compatibility Risks
13. Database and Security Risks
14. Step-by-Step Consolidation Plan
15. Final Recommendation

Serverless Function Consolidation Analysis

EXZIBO — Restaurant Management Platform
Vercel API Routes & Serverless Functions — Full Codebase Audit

Analysis-only report · No code modified · July 14, 2026

Serverless Function Consolidation Analysis (Read-only)

Scope: Analysis only. No code was modified, merged, deleted, or refactored as part of this report. **Date:** 2026-07-14

1. Executive Summary

The project currently deploys to Vercel as **12 serverless functions** under `api/*.js`, each a single Node.js handler that internally dispatches on an `action` query parameter (or sub-path, via `vercel.json` rewrites). This is already a resource-consolidated design — it is not “one function per CRUD operation.” The real consolidation opportunity is not merging these 12 files further; it is **removing the duplicate implementation that already exists** in two other places:

1. `vite.config.js` contains 8 Vite dev-server middleware plugins (`previewAuthPlugin`, `menuApiPlugin`, `aboutApiPlugin`, `restaurantDbPlugin`, `tableValidationPlugin`, `neonRestaurantPlugin`, `neonHealthPlugin`, `spaFallbackPlugin`) that **re-implement the same business logic** as `api/menu.js`, `api/orders.js`, `api/bookings.js`, `api/team.js`, `api/settings.js`, `api/content.js`, `api/media.js`, `api/restaurants.js`, and the preview-auth part of `api/system.js` — separately, for local dev only.
2. `server.js` (the `npm start` / production Express fallback server) independently re-implements a third copy of much of the same logic (preview auth, restaurant-db provisioning, table validation), including its own rate limiting and locking.

So the codebase effectively has **three parallel implementations** of most endpoints: the Vercel functions (`api/*.js`), the Vite dev middleware (`vite.config.js`), and the Express server (`server.js`). This is the highest-value, lowest-risk finding: consolidating dev/

runtime logic behind shared service modules (used by all three entry points) would remove the largest source of drift, without touching a single public API route.

Two frontend-called endpoints — `/api/analytics` and `/api/migrate` — have no corresponding `api/*.js` file and no `vercel.json` rewrite, so they are **broken/unreachable in production** (they would 404, or on the Vite dev server fall through to the SPA HTML fallback). `/api/migrate` is a client-called no-op in dev only; `/api/analytics` appears genuinely missing everywhere except a dev intent comment.

Within the 12 Vercel functions, authentication is applied **inconsistently**: `api/auth.js` and `api/auth-check.js` are properly session-gated; almost every other action (menu, content, media, orders, bookings, team, most of restaurants/settings) has **no server-side session check at all** and relies only on obscurity of restaurant IDs, CORS, and (for a few write paths) Upstash rate limiting. This is a security finding independent of any merge decision.

2. Current Serverless-Function Count

12 Vercel serverless functions (Node.js runtime, one per file in `api/`, each `export default async function handler`):

#	File	Public Path(s) (via <code>vercel.json</code>)
1	<code>api/auth.js</code>	<code>/api/auth/:path*</code>
2	<code>api/auth-check.js</code>	<code>/api/auth-check</code>
3	<code>api/restaurants.js</code>	<code>/api/neon/restaurants</code> , <code>/api/neon/restaurant/*</code> , <code>/api/restaurant/:id</code> , <code>/api/restaurant/update-profile</code> , <code>/api/restaurants?action=...</code>
4	<code>api/settings.js</code>	<code>/api/neon/restaurant-settings/shadow-upsert</code> , <code>/api/settings?action=...</code>
5	<code>api/team.js</code>	<code>/api/team-members/*</code>
6	<code>api/menu.js</code>	<code>/api/menu/*</code>
7	<code>api/content.js</code>	<code>/api/about/*</code> , <code>/api/restaurant/update-social</code>
8	<code>api/media.js</code>	<code>/api/menu/upload-image</code> , <code>/api/about/upload-image</code> , <code>/api/restaurant/upload-logo</code> , <code>/api/restaurant/upload-carousel</code>
9	<code>api/orders.js</code>	<code>/api/orders</code> , <code>/api/orders/:restaurantId</code> , <code>/api/orders/update-status</code> , <code>/api/orders/auto-cleanup</code>
10	<code>api/bookings.js</code>	<code>/api/bookings</code> , <code>/api/bookings/:restaurantId</code> , <code>/api/bookings/:id/status</code>
11	<code>api/notifications.js</code>	<code>/api/notifications?action=...</code>

#	File	Public Path(s) (via <code>vercel.json</code>)
12	<code>api/system.js</code>	<code>/api/preview-login</code> , <code>/api/preview-verify</code> , <code>/api/restaurant-db/*</code>

Plus `api/_lib/authz.js` and `api/_lib/cors.js` — shared utilities, **not** separate functions (no `export default handler`).

Not part of the Vercel deployment, but functionally overlapping: - `exzibo-realtime/` — a separate **Cloudflare Worker** (Durable Object, WebSocket Hibernation) for realtime order/booking events. Deployed independently via `wrangler`. - `server.js` — an Express server used only when running `npm start` (not part of the Vercel deployment, which uses `vercel.json` rewrites directly to `api/*.js`).

3. Current API and Serverless Directory Tree

```

api/
├── _lib/
│   ├── authz.js      (shared: session/role helpers – not a function)
│   └── cors.js       (shared: CORS headers – not a function)
├── auth.js           → Better Auth catch-all (/api/auth/:path*)
├── auth-check.js     → GET session/role verification
├── restaurants.js    → restaurant CRUD (action-routed)
├── settings.js       → global/user/restaurant settings (action-routed)
├── team.js           → restaurant_members CRUD
├── menu.js           → menu_categories + menu_items CRUD (action-routed)
├── content.js        → restaurant_about + social links (action-routed)
├── media.js          → Cloudflare R2 image uploads (action-routed)
├── orders.js         → orders CRUD + status + cleanup
├── bookings.js       → bookings CRUD + status
├── notifications.js  → messages/active_notification/notification_history/sms/help (action-routed)
└── system.js         → preview auth (HMAC) + restaurant-db schema provisioning (admin)

src/lib/
├── auth.server.js    → Better Auth server config (Google OAuth, schema bootstrap)
├── auth.client.js    → Better Auth browser client
├── r2.js             → Cloudflare R2 client (used only by api/media.js)
├── db.js             → single frontend API client – nearly every fetch() in the app goes through here
└── previewAuth.js    → frontend helper for /api/preview-login|verify

src/db/
├── schema.ts         → Drizzle schema (restaurants, restaurant_members, menu_categories, menu_items, orders, order_items, bookings, restaurant_about, restaurant_settings, table_numbers, audit_logs)
├── neon-restaurants.js, neon-restaurant-about.js, neon-globals.js, neon-restaurant-settings.js, pg-sql.js → query helpers used by api/*.js
├── drizzle/migrations/0004_add_global_tables.sql
│   └── raw-SQL tables NOT in schema.ts: global_settings, user_settings, messages, active_notification, notification_history, sms_notifications, help_notifications

vite.config.js (dev-only, NOT deployed to Vercel)
├── previewAuthPlugin() → duplicates api/system.js preview-login/verify
├── menuApiPlugin()     → duplicates api/menu.js + api/orders.js + api/bookings.js + api/team.js + api/settings.js

```

```

└─ aboutApiPlugin()      → duplicates api/content.js + api/media.js + parts of api/
restaurants.js
└─ restaurantDbPlugin()  → duplicates api/system.js restaurant-db provisioning
└─ tableValidationPlugin() → dev-only, mirrors table validation in server.js
└─ neonRestaurantPlugin() → duplicates api/restaurants.js (list/create/patch)
└─ neonHealthPlugin()    → simple health check, no Vercel equivalent (see §10)
└─ spaFallbackPlugin()   → dev-only SPA fallback (Vercel handled by vercel.json catch-all
rewrite)

server.js (Express; used only for `npm start`, not the Vercel deployment)
└─ Re-implements: preview auth, restaurant-db provisioning, table validation,
    with its own rate-limit/lock code – a THIRD copy of that logic.

exzibo-realtime/ (separate Cloudflare Worker deployment)
└─ src/index.ts – Durable Object WebSocket room for realtime order/booking push events.

```

How Vercel turns these files into functions

Each file in `api/` that has a default-exported handler becomes one independent Node.js serverless function. `vercel.json`'s `rewrites` do **not** create new functions — they only remap public URL paths (and REST-looking paths like `/api/menu/items/:restaurantId`) onto the existing `?action=` query-string interface of the 12 physical functions. This is why the count stays at 12 no matter how many public-looking REST paths are exposed.

The database migration (Supabase → Neon/Drizzle) did not create additional serverless functions. It only changed the database access code inside the existing functions (new query helpers in `src/db/neon-*.js`, replacing former Supabase client calls). Function count and boundaries are unchanged by that migration; only the DB layer changed.

4. Complete Serverless-Function Inventory

Full per-action detail (method, path, purpose, auth, params, response, DB tables, env vars, frontend callers) captured during this analysis. Abbreviated here per function; ask if you want the full per-action table expanded inline.

`api/auth.js`

- Method: ALL · Path: `/api/auth/:path*` · Runtime: Node.js

- Purpose: Better Auth catch-all (sign-in, callback, session, sign-out)
- Auth required: No (this is the auth entry point)
- DB: `user`, `session`, `account`, `verification` (bootstrapped via `ensureAuthSchema()` on every request)
- External services: Google OAuth
- Env vars: `BETTER_AUTH_BASE_URL`, `BETTER_AUTH_URL`, `BETTER_AUTH_SECRET`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `DATABASE_URL`, `BETTER_AUTH_TRUSTED_ORIGINS`
- Frontend: `src/lib/auth-client.js`, `src/lib/auth.server.js`
- Status: Active. Note: has a leftover diagnostic `console.log` for sign-in/callback with a TODO to remove it (`api/auth.js:29`).

`api/auth-check.js`

- Method: GET · Path: `/api/auth-check?type=superadmin|member&restaurantId=`
- Purpose: Fast role/permission check used by frontend routing
- Auth required: Yes (Better Auth session cookie; bypassed when `DISABLE_AUTH / VITE_DISABLE_AUTH=true`)
- DB: `restaurant_members`
- Env vars: `SUPERADMIN_ALLOWED_EMAILS`, `DATABASE_URL`, `DISABLE_AUTH`, `VITE_DISABLE_AUTH`
- Frontend: `src/context/AuthContext.jsx`, `src/pages/RestaurantDashboard.jsx`
- Status: Active.

`api/restaurants.js`

- Actions: `list`, `listDeleted`, `bySlug`, `byId`, `checkSlug`, `myIds`, `neonRestaurant` (GET/PATCH), `create`, `update` / `updateProfile`, `softDelete`, `permanentDelete`
- Auth required: Only `myIds` checks session; every other action (including `create`, `update`, `softDelete`, `permanentDelete`) has **no server-side auth check**
- DB: `restaurants` (+ cascading tables on delete)
- Frontend: `src/lib/db.js`, `src/pages/MasterControl.jsx`, `src/pages/DynamicRoute.jsx`, `src/pages/AddRolePage.jsx`

- Status: Active. `update` and `updateProfile` are redundant aliases of the same `patchNeonRestaurant` call.

`api/settings.js`

- Actions: `getGlobal` / `setGlobal` , `getUserSettings` / `saveUserSettings` (auth required), `getRestaurantSettings` , `setRestaurantSettings`
- DB: `global_settings` , `user_settings` , `restaurant_settings`
- Frontend: `src/lib/db.js` , `src/lib/routeConfig.js` , `src/lib/imageCompressionSettings.js`
- Status: Active.

`api/team.js`

- Actions: GET list, `create` / `update` / `shadowUpsert` , `delete` / `shadowDelete`
- DB: `restaurant_members`
- Auth required: No server-side check found
- Status: Active.

`api/menu.js`

- Actions: `getCategories` , `upsertCategory` , `deleteCategory` , `getItems` , `getPublishedItems` , `createItem` , `updateItem` , `upsertItems` , `deleteItem`
- DB: `menu_categories` , `menu_items`
- Auth required: No server-side check on any action
- Frontend: `src/lib/db.js` , `src/pages/AdminDashboard.jsx` , `src/pages/FoodDetail.jsx`
- Status: Active. **Known upsert-vs-patch hazard** — `updateItem` (`/api/menu/item-patch`) is a full upsert; sending a partial payload blanks other columns (already a documented gotcha for this project).

`api/content.js`

- Actions: `getAbout` , `saveAbout` , `updateSocial`
- DB: `restaurant_about` , `restaurants`
- Auth required: None

- Status: Active.

`api/media.js`

- Actions: `uploadMenuImage` , `uploadAboutImage` , `uploadLogoImage` , `uploadCarouselImage` — all POST, all write to Cloudflare R2
- External services: Cloudflare R2, Upstash rate limiting
- Env vars: `R2_ACCOUNT_ID` , `R2_ACCESS_KEY_ID` , `R2_SECRET_ACCESS_KEY` , `R2_BUCKET_NAME` , `R2_PUBLIC_BASE_URL` , `UPSTASH_REDIS_REST_URL` , `UPSTASH_REDIS_REST_TOKEN`
- Auth required: None (rate-limited only)
- Status: Active.

`api/orders.js`

- Actions: GET list, POST create, `update-status` , `auto-cleanup`
- DB: `orders`
- External services: Upstash (rate limit + lock on `update-status`), realtime event dispatch (to the Cloudflare Worker)
- Auth required: None
- Frontend: `src/pages/RestaurantWebsite.jsx` , `src/lib/db.js` , `src/lib/orderCleanup.js`
- Status: Active. `auto-cleanup` behaves like an admin/cron-style maintenance action but is triggered client-side from `src/lib/orderCleanup.js` , not by an actual Vercel Cron job.

`api/bookings.js`

- Actions: GET list, POST create, `PATCH :id/status`
- DB: `bookings`
- External services: Upstash (rate limit + lock on all writes)
- Auth required: None
- Status: Active.

`api/notifications.js`

- Actions: `sendMessage` , `getMessages` , `getActiveNotification` , `publishActiveNotification` , `confirmActiveNotification` , `insertNotificationHistory` , `getNotificationHistory` ,

- `getLatestSms`, `upsertSms`, `createHelp`, `getHelp`,
`updateHelpStatus`, `deleteHelp`, `markAllHelpRead`
- DB: `messages`, `active_notification`, `notification_history`,
`sms_notifications`, `help_notifications` (all from `drizzle/`
`migrations/0004_add_global_tables.sql`, **not** in `schema.ts`)
- Auth required: None found — several actions
(`publishActiveNotification`, `insertNotificationHistory`,
`upsertSms`, help-desk status changes) look admin-only by intent
but have no role check
- Status: Active.

`api/system.js`

- Actions: `preview-login` (POST), `preview-verify` (GET),
`restaurant-db/create` (POST), `restaurant-db/drop` (POST),
`restaurant-db/list` (GET)
- Purpose: HMAC-based preview-mode login, and per-restaurant
Postgres schema provisioning (a separate, older multi-tenancy
mechanism from the main `restaurants` table model)
- Env vars: `PREVIEW_EMAIL`, `PREVIEW_PASSWORD_HASH`,
`PREVIEW_SECRET`, `REPL_ID`, `DATABASE_URL`
- Auth required: `restaurant-db/*` has no visible role gate —
should be superadmin-only given it can
`DROP SCHEMA ... CASCADE`
- Status: Active, but see §9 — this “per-restaurant Postgres
schema” provisioning model looks like a separate, possibly
legacy multi-tenancy approach that coexists with the
`restaurants` / `restaurant_members` row-based model. Needs a
product decision, not a code merge.

**Broken / missing (called by frontend, no backend implementation
reaches production)**

- `/api/analytics?action=orderCountThisMonth` /
`restaurantsCreatedThisMonth` — called from `src/lib/db.js`. No
`api/analytics.js` file, no `vercel.json` rewrite, no vite dev-
middleware handler, no `server.js` handler. **Broken in every
environment.**

- `/api/migrate` — called from `src/pages/AdminDashboard.jsx`. Only handled in `vite.config.js` (explicit no-op, comment says “Supabase migration removed”). No `api/migrate.js`, no `vercel.json` rewrite. **404s in production**. Low risk since it’s already a no-op, but it’s dead code calling a dead endpoint.

5. Frontend-to-API Dependency Map

Nearly all frontend calls are centralized in `src/lib/db.js`, which acts as the app’s API client (`fetch` / `apiFetch` wrapper). A handful of pages call `fetch` directly for specific flows:

Frontend file	Calls
<code>src/lib/db.js</code>	The large majority of all <code>/api/*</code> calls (restaurants, settings, team, menu, content, media, orders, bookings, notifications)
<code>src/pages/RestaurantWebsite.jsx</code>	<code>/api/orders</code> , <code>/api/orders/:id</code> , <code>/api/bookings</code> (customer-facing ordering/booking flow)
<code>src/pages/RestaurantDashboard.jsx</code>	<code>/api/auth-check?type=member</code>
<code>src/context/AuthContext.jsx</code>	<code>/api/auth-check?type=superadmin</code>
<code>src/pages/MasterControl.jsx</code> , <code>src/pages/AddRolePage.jsx</code> , <code>src/pages/DynamicRoute.jsx</code>	<code>/api/neon/restaurant*</code> (superadmin restaurant management)
<code>src/pages/FoodDetail.jsx</code>	<code>/api/menu/items/:restaurantId</code>
<code>src/pages/AdminDashboard.jsx</code>	<code>/api/migrate</code> (dead call, see §4)
<code>src/lib/orderCleanup.js</code>	<code>/api/orders/auto-cleanup</code>
<code>src/lib/previewAuth.js</code>	<code>/api/preview-login</code> , <code>/api/preview-verify</code>
<code>src/lib/routeConfig.js</code> , <code>src/lib/imageCompressionSettings.js</code>	<code>/api/settings?action=getGlobal/setGlobal</code>
<code>src/pages/NotificationsPage.jsx</code> , <code>src/components/Sidebar.jsx</code> , <code>src/components/NotificationDrawer.jsx</code> , <code>src/components/HelpRequestsDrawer.jsx</code>	<code>/api/notifications?action=...</code>
<code>src/hooks/useRealtimeOrders.js</code>	WebSocket to the separate Cloudflare Worker (<code>exzibo-realtime/</code>), not a Vercel API route

6. Duplicate and Overlapping Function Analysis

The 12 Vercel functions themselves have **very little duplication with each other** — each owns a distinct resource/table. The duplication is between the Vercel functions and the two other server entry points:

Public logic	Vercel function (production)	Vite dev middleware (<code>vite.config.js</code>)	Express (<code>server.js</code>)
Menu CRUD	<code>api/menu.js</code>	<code>menuApiPlugin()</code>	—

Public logic	Vercel function (production)	Vite dev middleware (vite.config.js)	Express (server.js)
Orders/Bookings/Team	api/orders.js , api/bookings.js , api/team.js	menuApiPlugin() (same plugin, multiple concerns)	—
About/uploads	api/content.js , api/media.js	aboutApiPlugin()	—
Restaurant list/create/patch	api/restaurants.js	neonRestaurantPlugin()	—
Preview auth (HMAC)	api/system.js	previewAuthPlugin()	server.js lines ~268-311
Restaurant-DB schema provisioning	api/system.js	restaurantDbPlugin()	server.js lines ~315-461
Table-number validation	(none in Vercel — dev/prod-server only feature)	tableValidationPlugin()	server.js lines ~97-164
Health check	(none in Vercel)	neonHealthPlugin() (/ api/health/neon)	—

Additionally, within `api/restaurants.js`, the `update` and `updateProfile` actions both call `patchNeonRestaurant` with no functional difference — a same-file redundancy, not a route-level one.

7. Functions That Can Be Merged

None of the 12 public Vercel functions should be merged with each other. They are already resource-scoped, and further merging (e.g. combining `orders.js` and `bookings.js` into one “operations” function) would violate the stated rule against combining unrelated business operations, and would not reduce real complexity — it would just make one file larger.

The only same-file, same-route redundancy found:

Functions involved	File	API path	Why duplicated	Recommendation	Risk
<code>update</code> action vs <code>updateProfile</code> action	<code>api/restaurants.js</code>	Both ultimately reach <code>/api/restaurant/update-profile</code> via <code>patchNeonRestaurant</code>	Two action names for the same operation, likely left over from an earlier refactor	Keep one canonical action name (<code>updateProfile</code>), deprecate <code>update</code> after confirming no caller uses it	Low — pure cleanup, verify zero frontend callers of the <code>update</code> action

Functions involved	File	API path	Why duplicated	Recommendation	Risk
					name first

8. Functions That Should Share Logic But Remain Separate

This is the primary opportunity in this codebase — **Classification B** (keep routes separate, share backend code):

Area	Current duplication	Shared logic to extract	Where it would live	Consumers	Risk
Preview auth (HMAC sign/verify)	Implemented independently in <code>api/system.js</code> , <code>vite.config.js</code> (<code>previewAuthPlugin</code>), and <code>server.js</code>	<code>signPreviewToken()</code> / <code>verifyPreviewToken()</code>	New <code>src/lib/previewAuthCore.js</code> (or similar), imported by all three	<code>api/system.js</code> , <code>vite.config.js</code> , <code>server.js</code>	Low — pure extraction, same input/output contracts
Restaurant-DB schema provisioning (create/drop/list per-tenant Postgres schema)	Implemented independently in <code>api/system.js</code> , <code>vite.config.js</code> (<code>restaurantDbPlugin</code>), and <code>server.js</code>	<code>createRestaurantSchema()</code> , <code>dropRestaurantSchema()</code> , <code>listRestaurantSchemas()</code>	New <code>src/db/restaurant-schema-provisioning.js</code>	Same three	Medium — touches DDL (<code>CREATE/DROP SCHEMA</code>); test drop path carefully in a non-prod DB first
Menu/content/media CRUD helpers	<code>api/menu.js</code> , <code>api/content.js</code> , <code>api/media.js</code> already delegate to <code>src/db/neon-*.js</code> helpers in production; the vite dev plugins re-implement the SQL inline instead of calling those same helpers	Point <code>menuApiPlugin</code> / <code>aboutApiPlugin</code> at the existing <code>src/db/neon-*.js</code> functions instead of duplicating SQL	No new file — reuse what <code>api/*.js</code> already imports	<code>vite.config.js</code>	Medium — must confirm the dev plugins' inline SQL doesn't intentionally differ (e.g. different validation) before pointing them at the shared helpers
Auth/role checks	<code>api/_lib/authz.js</code> already centralizes this well for <code>api/auth-check.js</code> ; most other functions (<code>restaurants</code> , <code>menu</code> , <code>orders</code> , <code>bookings</code> , <code>team</code> , <code>content</code> , <code>notifications</code>) don't call it at all	Have every write-action function call <code>requireSession</code> / <code>requireRestaurantAccess</code> from the existing <code>api/_lib/authz.js</code>	Existing file, wire it into the other 8 functions	All 12 functions	Medium-High — this changes real authorization behavior (currently many actions are effectively public); needs product sign-off on which roles should gate which action before

Area	Current duplication	Shared logic to extract	Where it would live	Consumers	Risk
					implementing, and must be rolled out function-by-function with testing, not all at once
Health check	<code>neonHealthPlugin</code> (<code>/api/health/neon</code>) exists only in dev; no equivalent exists in <code>api/</code> for production monitoring	Add one lightweight <code>api/health.js</code> used by both dev and prod (or extract the <code>SELECT 1</code> check into a shared helper)	New <code>api/health.js</code> (this would be a genuinely new, 13th function — separate decision)	Uptime/monitoring	Low

9. Functions That Must Not Be Merged

Function/area	Why it must stay separate
<code>api/auth.js</code> (Better Auth catch-all)	Handles OAuth redirect/callback state, session cookie issuance, and CSRF protections that are tightly coupled to Better Auth's own routing contract. Merging it with any other handler would break Better Auth's internal path matching and its cookie/session lifecycle.
<code>api/auth-check.js</code>	Pure authorization check with its own caching/response contract consumed synchronously by the frontend router before rendering. Merging it into a business-logic function would couple authenticating to unrelated read/write operations and complicate caching.
Google OAuth flow (inside <code>api/auth.js</code>)	OAuth callback URLs are registered with the external provider (Google) and must remain stable, dedicated endpoints — moving this logic elsewhere breaks the registered redirect URI.
<code>exzibo-realtime/</code> Cloudflare Worker (WebSocket/Durable Object)	Long-lived stateful WebSocket connections cannot run on Vercel's stateless serverless functions at all (no persistent connections, no Durable Object equivalent). This must remain a separate Cloudflare deployment regardless of any Vercel-side consolidation.
R2 upload logic (<code>api/media.js</code>)	Uses direct binary/base64 handling and Cloudflare R2 credentials; keeping it isolated limits the blast radius of storage credentials and avoids mixing binary upload handling with JSON CRUD handlers (different request/response shapes, different failure modes).
<code>api/system.js</code> restaurant-db provisioning (<code>create</code> / <code>drop</code>)	Executes raw DDL (<code>CREATE SCHEMA</code> , <code>DROP SCHEMA ... CASCADE</code>) — an operation with a fundamentally different risk profile (irreversible data loss) than any CRUD action. It must remain gated and isolated, not folded into a general-purpose function where a validation slip could reach it.

Function/area	Why it must stay separate
Preview-login/verify (<code>api/system.js</code>)	A separate, lower-trust authentication path (HMAC token, not session-based) used only for demo/preview access. Combining it with the main Better Auth flow would blur two different trust boundaries.
Superadmin-only vs. restaurant-scoped actions (e.g. <code>listDeleted</code> , <code>permanentDelete</code> in <code>restaurants.js</code> vs. regular <code>list</code> / <code>byId</code>)	Even within one file, these already mix permission boundaries (see §13) — a further merge across files would make that worse, not better. Any future split should go the other direction (separate superadmin actions out), not merge more in.
<code>orders.js</code> / <code>bookings.js</code> write paths using Upstash locks	The lock/rate-limit keys are scoped per-operation; merging endpoints risks colliding lock keys or changing the idempotency window unintentionally.

10. Separate Route Merging From Code Deduplication

Classification applied to every finding in this report:

- **A. Merge API routes:** Only the `update` / `updateProfile` alias inside `api/restaurants.js` (§7). No cross-file public route merges are recommended.
- **B. Keep API routes separate but share backend logic:** Preview auth, restaurant-db provisioning, menu/content/media SQL helpers, auth/role checks, health check (§8).
- **C. Remove obsolete function:** `/api/analytics` (broken, no backend anywhere — either implement it for real or delete the two call sites in `src/lib/db.js`); `/api/migrate` (dead no-op call in `src/pages/AdminDashboard.jsx`, backed only by a dev no-op — safe to delete the call site and the vite handler together).
- **D. Keep unchanged:** All 12 Vercel functions as public route boundaries; the Cloudflare Worker; the Better Auth and OAuth handlers; the R2 upload handler; the restaurant-db DDL provisioning as a route (its implementation can still be shared per §8).

11. Recommended Final API Structure

No public route renaming is recommended — the existing `?action=` resource-grouped structure (`/api/restaurants` , `/api/menu` , `/api/orders` , `/api/bookings` , `/api/settings` , `/api/team` , `/api/content` , `/api/media` , `/api/notifications` , `/api/system` , `/api/auth` , `/api/auth-check`) already matches the intent of a clean, resource-based tree, just addressed via query-string actions instead of path segments. Renaming paths (e.g. to `/api/restaurants/[id]/menu/items`) would require rewriting `vercel.json` , every `fetch()` call site in `src/lib/db.js` , and re-testing every flow — a large-surface change for a purely cosmetic improvement, not recommended unless there’s a separate reason (e.g. adopting Next.js) to do it.

If restructuring is ever desired, the natural resource tree (for future reference, not a proposal to implement now) would be:

<code>/api/auth/...</code>	(unchanged – Better Auth)
<code>/api/auth-check</code>	(unchanged)
<code>/api/restaurants</code>	→ <code>api/restaurants.js</code> (list/create)
<code>/api/restaurants/:id</code>	→ <code>api/restaurants.js</code> (get/patch/delete)
<code>/api/restaurants/:id/members</code>	→ <code>api/team.js</code>
<code>/api/restaurants/:id/menu/categories</code>	→ <code>api/menu.js</code>
<code>/api/restaurants/:id/menu/items</code>	→ <code>api/menu.js</code>
<code>/api/restaurants/:id/orders</code>	→ <code>api/orders.js</code>
<code>/api/restaurants/:id/orders/:orderId</code>	→ <code>api/orders.js</code>
<code>/api/restaurants/:id/bookings</code>	→ <code>api/bookings.js</code>
<code>/api/restaurants/:id/bookings/:bookingId</code>	→ <code>api/bookings.js</code>
<code>/api/restaurants/:id/settings</code>	→ <code>api/settings.js</code>
<code>/api/restaurants/:id/about</code>	→ <code>api/content.js</code>
<code>/api/uploads/...</code>	→ <code>api/media.js</code>
<code>/api/notifications/...</code>	→ <code>api/notifications.js</code>
<code>/api/system/...</code>	→ <code>api/system.js</code>
<code>/api/health</code>	→ new, extracted from <code>vite-only neonHealthPlugin</code>

This is descriptive, not a migration instruction — implementing it is a separate, larger project (would touch every `fetch()` call site) and should be its own proposed task if wanted.

12. Vercel Serverless Compatibility Risks

Function	Risk found	Safe for Vercel?
<code>api/auth.js</code>	Runs <code>ensureAuthSchema()</code> (DDL bootstrap) on every request	Works, but adds latency to every auth request; low risk, could be memoized per cold-start instead of per-request

Function	Risk found	Safe for Vercel?
<code>api/orders.js</code> , <code>api/bookings.js</code>	Upstash-based locks (5s/300s) used for idempotency	Correctly designed for serverless (external lock store, not in-memory) — safe
<code>api/system.js</code> <code>restaurant-db/create / drop</code>	Raw multi-statement DDL in a single request	No timeout handling visible; large schema creation could approach Vercel's function timeout under load — low likelihood given per-tenant table count, but worth monitoring
<code>api/media.js</code>	Handles <code>dataUrl</code> (base64) uploads directly in the function body	Base64 payloads inflate request size ~33%; fine for typical images but could hit Vercel's request body size limit for very large images — no explicit size validation found
None found	In-memory state, persistent WebSockets, or background processing after response	Not present in any <code>api/*.js</code> file — correctly avoided
<code>exzibo-realtime/</code>	WebSocket Durable Object	Correctly kept outside Vercel (Cloudflare Workers), as required — not a Vercel compatibility risk since it isn't deployed there

No function contains Cloudflare-specific code (Durable Objects, Workers KV) — R2 access uses the standard S3-compatible API via `src/lib/r2.js`, which is portable and fine to run inside a Vercel Node function.

13. Database and Security Risks

- **Restaurant isolation:** Enforced only by requiring a `restaurantId / restaurant_id` parameter in the request body/query — there is **no server-side check that the caller is actually authorized for that restaurant** on most write actions (menu, content, media, orders, bookings, team, most of restaurants/settings). Any client that knows or guesses a restaurant UUID can currently read or write its data. This is a pre-existing risk, not something a merge would introduce, but any consolidation work in §8 should fix this at the same time by wiring in `requireRestaurantAccess` from `api/_lib/authz.js`.
- **Superadmin-only operations without a role check:** `api/system.js`'s `restaurant-db/create|drop|list` and several `api/notifications.js` admin actions (`publishActiveNotification` ,

`insertNotificationHistory` , `upsertSms`) have no visible auth gate, despite being destructive or broadcast-level operations.

- **`softDelete` / `permanentDelete` / `update` / `create` in `api/restaurants.js`** : No session check — same class of risk as above, but on the primary tenant table.
- **Neon connection handling**: Each function creates its own `neon(process.env.DATABASE_URL)` client per request via the serverless (HTTP-based) Neon driver — this is the correct serverless-safe pattern (no persistent pool exhaustion risk).
- **Drizzle vs. raw SQL split**: `schema.ts` /Drizzle covers the core tenant tables; `global_settings` , `user_settings` , `messages` , `active_notification` , `notification_history` , `sms_notifications` , `help_notifications` are raw-SQL tables from `drizzle/migrations/0004_add_global_tables.sql` and are **not** tracked by `drizzle-kit push` . Any future `db:push` will not manage these tables — schema drift here would go undetected by the normal Drizzle workflow.
- **Duplicate-write protection**: Present for order status updates and bookings (Upstash lock), consistent with expectations; not present for menu item writes (not usually needed) or restaurant profile writes (lower risk, single-row updates).
- **No cross-restaurant access became newly possible in this analysis** — this is a pre-existing gap; no merge recommended here would change that exposure either way.

14. Step-by-Step Consolidation Plan

Each step is independently shippable and rollback-able. No step in this plan changes any public API path.

Step 1 — Remove confirmed dead/obsolete code

(Classification C) - Files to inspect: `src/lib/db.js` (the two `/api/analytics?...` calls), `src/pages/AdminDashboard.jsx` (`/api/migrate` call), `vite.config.js` (`/api/migrate` no-op handler) -

Recommended change: Either implement `/api/analytics` for real (if the dashboard metric is wanted) or delete the two call sites and any UI that depends on them; delete the `/api/migrate` call site and its vite no-op handler together - Frontend callers affected: `src/lib/`

`db.js`, `src/pages/AdminDashboard.jsx` - Tests required: Confirm the analytics UI element (if any) is not silently broken already; confirm no other code imports the removed functions - Rollback condition: If analytics numbers are needed elsewhere, restore and implement instead of deleting - Risk: Low

Step 2 — Extract shared preview-auth logic (Classification B)

- Files to inspect: `api/system.js`, `vite.config.js` (`previewAuthPlugin`), `server.js` (~lines 268-311) - Recommended change: Extract HMAC sign/verify into one shared module; have all three call sites use it - Frontend callers affected: None (internal refactor only, same request/response contract) - Tests required: Manual preview-login flow in dev, staging, and prod - Rollback condition: Any auth-token mismatch between environments after extraction - Risk: Low

Step 3 — Extract shared restaurant-db provisioning logic (Classification B)

- Files to inspect: `api/system.js`, `vite.config.js` (`restaurantDbPlugin`), `server.js` (~lines 315-461) - Recommended change: Extract `createRestaurantSchema` / `dropRestaurantSchema` / `listRestaurantSchemas` into one shared module - Frontend callers affected: None directly (internal), but this powers `src/lib/db.js` restaurant-db calls - Tests required: Create + drop a test schema in a non-production database before relying on it in prod - Rollback condition: Any DDL error difference between the three current implementations surfaces during testing - Risk: Medium (DDL, including `DROP SCHEMA CASCADE`)

Step 4 — Point Vite dev plugins at existing `src/db/neon-*.js` helpers instead of inline SQL (Classification B)

- Files to inspect: `vite.config.js` (`menuApiPlugin`, `aboutApiPlugin`, `neonRestaurantPlugin`), `src/db/neon-restaurants.js`, `src/db/neon-restaurant-about.js` - Recommended change: Replace inline SQL in dev plugins with calls to the same helpers `api/*.js` already uses - Frontend callers affected: None (dev server behavior only; must match existing prod behavior exactly) - Tests required: Full manual regression of menu/about/restaurant CRUD in dev after the change - Rollback condition: Any behavior difference between old inline SQL and the shared helper is discovered - Risk: Medium

Step 5 — Merge the one same-file redundant action

(Classification A) - Files to inspect: `api/restaurants.js`, `src/lib/db.js` - Recommended change: Confirm zero callers use the `update` action name (only `updateProfile`); if confirmed, remove the redundant branch - Frontend callers affected: Confirm via grep before removing - Tests required: Grep-based confirmation + one manual profile-update test - Rollback condition: Any caller found using `action=update` - Risk: Low

Step 6 — Add authentication/authorization checks (security fix, not a merge)

- Files to inspect: `api/_lib/authz.js`, then one function at a time: `api/restaurants.js`, `api/menu.js`, `api/content.js`, `api/media.js`, `api/orders.js`, `api/bookings.js`, `api/team.js`, `api/settings.js`, `api/notifications.js`, `api/system.js` - Recommended change: Wire `requireSession` / `requireRestaurantAccess` into each write action, matched to the correct role per action (needs a product decision on required roles per action — do not guess) - Frontend callers affected: Every write flow in the app — must be rolled out and tested function-by-function, not all at once - Tests required: Full auth-flow regression per function as it's updated - Rollback condition: Any legitimate flow starts failing with 401/403 after a given function's rollout - Risk: **High** — this is a genuine behavior change (currently-open endpoints become gated) and needs explicit product sign-off before starting, separate from this analysis

Step 7 — Verify production deployment behavior - After Steps 1-5 (low/medium risk), redeploy to Vercel preview environment and smoke-test every public path listed in §4 - Step 6 (auth hardening) should get its own dedicated staged rollout and sign-off, independent of this consolidation work

15. Final Recommendation

Do not merge any of the 12 existing Vercel functions with each other — they are already correctly resource-scoped, and further merging would violate the stated goals (no combining unrelated operations, no generic mega-endpoint, no merging public/protected boundaries). The actual, high-value consolidation

opportunity is **eliminating the duplicate re-implementation of the same logic across** `vite.config.js`, `server.js`, and `api/*.js` by extracting shared modules (Steps 2-4 above) — this reduces real drift risk without touching any public contract. Separately, but not part of “consolidation,” this analysis surfaced two concrete defects worth fixing on their own track: the missing `/api/analytics` backend (Step 1) and the near-total absence of server-side authorization checks outside of `api/auth.js` / `api/auth-check.js` (Step 6) — the latter is a security gap, not a structural one, and should be scoped and approved as its own piece of work.